



Correction de la Série d'Exercices TD/TP 03

Module : Algorithmique et Python 2

Filière : MIP

Semestre : S2

Etablissement : FSSM

Année Universitaire : 2023/2024

Corrigé par : Passerelle, Centre de Formation et d'Orientation

Adresse : LOT SIDI ABBAD 1 IMM N 298 ETG 3 BUR N 16

Email : info@passerelle.ma

Téléphone : +212 666 92 44 62 / +212 524 30 73 09

Introduction

Chers étudiants,

Ce document présente la correction de la série d'exercices TD/TP 3 du module "Algorithmique et Python 2" de la filière MIP, semestre S2, pour l'année universitaire 2023/2024, à la Faculté des Sciences Semlalia de Marrakech (FSSM).

L'objectif de cette série d'exercices est de vous familiariser avec les concepts de récursivité en Python, la manipulation des listes, et l'approfondissement de votre compréhension des structures de contrôle itératives et conditionnelles. Vous aborderez également des notions essentielles telles que la distinction entre les fonctions itératives et récursives à travers des exemples spécifiques qui renforceront vos compétences en programmation et vous aideront à comprendre les mécanismes internes du langage Python.

Les corrections fournies dans ce document comprennent des explications détaillées pour chaque question, illustrées par des exemples de code et des descriptions des démarches nécessaires pour résoudre les problèmes proposés. Cela vous permettra de mieux saisir les concepts abordés et de développer votre capacité à appliquer ces connaissances dans des contextes pratiques.

Nous espérons que ce document vous sera utile dans votre apprentissage et qu'il vous aidera à maîtriser les sujets traités. N'hésitez pas à contacter votre professeur pour toute question ou clarification supplémentaire.

Bonne lecture et bon travail !

Professeur : ER-RAHMOUNY Hamza.

Exercice 01 :

1. Qu'est-ce que la récursivité, que doit-t-on définir dans une fonction récursive

La récursivité est une technique en programmation où une fonction s'appelle elle-même pour résoudre un problème plus complexe en le divisant en sous-problèmes plus simples.

Dans une fonction récursive, il est essentiel de définir deux éléments :

_ Le cas de base : C'est la condition qui arrête la récursion. Sans cas de base, la fonction récursive entrerait dans une boucle infinie.

_ L'appel récursif : C'est la partie où la fonction s'appelle elle-même avec des arguments modifiés pour se rapprocher du cas de base.

2. Ecrivez une fonction qui fait la multiplication de deux nombres, par la méthode itérative, et par la méthode récursive. Ecrire le suivi de l'exécution pas à pas de votre programme.

Remarque : $a * b = a + a + \dots + a$ (b fois)

_ Fonction multiplication par la méthode récursive :

```
def multiplication_réursive(a, b) :  
    if (b == 1) :      #Si b = 1, donc on a effectué a + a (b-1 fois) donc il ne reste qu'ajouter un seul a.  
        return a  
    else :             #Sinon il faut encore faire appel à la fonction pour faire la somme b-1 fois.  
        return a + multiplication_réursive(a, b-1)
```

_ Suivi de la fonction multiplication_réursive pour les valeurs a = 9 et b = 3

Le principe ici est le suivant : Chaque fonction doit soit retourner 9 soit retourner 9 + appel de la fonction avec b-1.

Donc nous avons le suivi ci-dessous :

~~multiplication_réursive(9,3)~~ 9 + ~~multiplication_réursive(9,2)~~ 9 + ~~multiplication_réursive(9,1)~~ 9

_ Fonction de la multiplication par la méthode itérative

```
def multiplication_itérative(a, b) :  
    s = 0                #Initialiser s par 0, pour l'utiliser pour cumuler la somme des a b fois.  
    for i in range(b) :  #La boucle va boucler b fois, et les valeurs de i ne sont pas importantes.  
        s = s + a        #s va cumuler cette somme : a + a + ... + a (b fois).  
    return s
```

_ Suivi de la fonction multiplication_itérative pour les valeurs a = 9 et b = 3

s = 0

- 1ère itération externe (i=0) :
s = 0 + 9
- 2ème itération externe (i=1) :
s = 9 + 9
- 3ème itération externe (i=2) :
s = 9 + 9 + 9

return 9 + 9 + 9

Exercice 02 :

Ecrivez une fonction qui calcule le factoriel d'un nombre, par la méthode itérative, et par la méthode récursive. Ecrire le suivi de l'exécution pas à pas de votre programme.

_ Fonction factorielle par la méthode récursive :

```
def fact_réursive(n) :
```

```
    if ( n < 0 ) :
```

```
        return None
```

```
        #Si n < 0, n! n'est pas défini, donc retourner rien.
```

```
    if ( n == 0 or n == 1 ) :
```

```
        return 1
```

```
        #Si n = 0 ou n = 1, n! = 1 par définition.
```

```
    return n * fact_réursive(n - 1)
```

```
    #Si n>1, par définition n! = n * (n-1)!
```

_ Suivi de la fonction fact_réursive(n) pour la valeur n = 3

Le principe ici est le suivant : Chaque fonction doit soit retourner None soit retourner 1 soit retourner 3 * appel de la fonction avec n-1.

Donc nous avons le suivi ci-dessous :

~~fact_réursive(3)~~ 3 * ~~fact_réursive(2)~~ 2 * ~~fact_réursive(1)~~ 1 = 3 * 2 * 1 = 3!

_ Fonction factorielle par la méthode itérative :

```
def fact_itérative(n) :  
    F = 1                                #Initialiser F pour cumuler les produits.  
    for i in range (1, n+1) :            # i va prendre ces valeurs (1, 2, ..., n)  
        F = F * i                        # F va cumuler 1 * 2 * ... * n  
    return F
```

_ Suivi de la fonction fact_itérative(n) pour la valeur n = 3

F = 1

- 1ère itération externe (i=1) :
F = 1 * 1
- 2ème itération externe (i=2) :
F = 1 * 1 * 2
- 3ème itération externe (i=3) :
F = 1 * 1 * 2 * 3

return 1 * 1 * 2 * 3

Exercice 03 :

Ecrivez une fonction qui prend en paramètre un indice et calcule le terme correspondant de la série de Fibonacci.

Exemple : 0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + ...

Indice = 6, terme = 8, Indice = 4, terme = 3

a) Par la méthode itérative

```
def Fibo(n) :  
    if (n <= 1) :                        #Dans Fibonacci le terme 0 est égal à 0 et le terme 1 est égal à 1.  
        return n  
    U0 = 0                              #Initialisation du terme 0 de Fibonacci par 0.  
    U1 = 1                              #Initialisation du terme 1 de Fibonacci par 1.  
    for i in range(n-1) :                # for va boucler seulement n-2 fois car déjà deux termes sont initialisés  
        temp = U1                        #Garder la valeur de U1 dans la variable temporaire temp.  
        U1 = U1 + U0                    #Changer la valeur de U1 par le terme suivant de la suite de Fibonacci.  
        U0 = temp                       #Changer la valeur de U0 par l'ancien valeur de U1.  
    Return U1
```

b) Par la méthode récursive naïve

```
def Fibo(n) :  
    if ( n <= 1 ) :                #Fibo(0) = 0 et Fibo(1) = 1.  
        return n  
    else :  
        return Fibo (n-1) + Fibo(n-2)    #Si n > 1, Fibo(n) = Fibo(n-1) + Fibo(n-2).
```

c) Par la méthode récursive avec mémorisation

```
d = {}                #Initialisation d'un dictionnaire vide en dehors de la fonction récursive !  
def Fibo(n) :  
    if n in d :        #Si n existe dans d déjà, retourner sa valeur pour éviter la calculer à nouveau.  
        return d[n]  
    if n <= 1 :        #Fibo(1) = 1 et Fibo(0) = 0  
        return n  
    d[n] = Fibo(n-1) + Fibo(n-2)    #si n>1 et n'existe pas déjà dans d calculer et stocker sa valeur  
    return d[n]
```

d) Pourquoi utilise-t-on la mémorisation

La mémorisation est une technique d'optimisation utilisée principalement pour accélérer les fonctions récursives en stockant les résultats des appels de fonction précédents pour éviter les appels récursives redondants.

Exercice 04 :

Ecrivez une fonction qui vérifie si une chaîne de caractère est palindrome (utiliser une approche récursive).

Note : Palindrome est une chaîne de caractère qui est lue de gauche à droite de même manière quand est lue de droite à gauche.

Exemple : radar

```
def palind(s) :  
    if (len(s) <= 1) :            #Si on arrive à un mot qui contient un seul caractère donc c'est palindrome.  
        return True  
    return s[0] == s[-1] and palind(s[1:-1])  
  
    #Si s contient plusieurs caractères, on compare le premier caractère avec le dernier, et on appelle la fonction  
    pour faire le même travail sur les éléments du s à partir du deuxième élément jusqu'à l'avant dernier élément.
```